Claude Prompt:

Build a training and inference simulator CPU code using MPI and C++. The core kernel used in training simulation is DGEMM (use BLAS, assume openblas), and the training loop should run across fixed #epochs, within a loop there should be one DGEMM operation corresponding to forward pass, two backward passes (one for gradients w.r.t weights and the other for gradients w.r.t inputs) and a parameter update via DAXPY. The core kernel in inference simulation is DGEMV (use BLAS, assume openblas), and the inference workload would be processing a batch of single-vector inputs through multiple layers, applying weights, biases, and a ReLU activation function. For both training and inference workload, assume randomly generated 100 shape combinations, maximum size of a dimension not exceeding 128. Create an MPI based harness to continuously schedule the training and inference simulation tasks/kernels across the available processes (assume odd or even number of processes). Create a global task counter (i.e., NXTVAL) using MPI-3 RMA, specifically MPI_Fetch_and_op to atomically fetch current counter value and add 1, to assign training or inference tasks to processes equitably within an infinite loop (you can maintain a counter per process such as inference_tasks and training_tasks and occasionally sync with other processes to adjust the workload). Also, in that infinite loop, have a separate thread continuously check argc for a floating point argument called priority (0-1, initial value is 0.5) which determines the probability of choosing inference or training simulation kernel/task (initially 50-50). If the priority changes to favor inference tasks (let's say from p:0.5 to p:0.9), then all current training tasks across processes should immediately save the partial computations into a checkpoint before taking up an inference task. Also, in the infinite while loop, check for another argument that refers to task/process suspension (like if there are 4 processes, and if we pass s:1, then rank/process \#1 must save training/inference checkpoint to a specific file and suspend from taking up new tasks until we pass r:1; when r:1 is passed, a process first checks if it has leftover tasks in the form of a checkpoint, and only takes up new tasks after existing checkpoints are consumed). If we pass "e:0" while the simulation is ongoing, it means "break out of the infinite loop without saving checkpoints" in whatever state you are in. Contrastingly, "e:1" means "break out of the infinite loop but save checkpoints". For each successful task run, increment the flop count per process/rank (keeping training and inference flops separate), and upon exit report it, such that per-process/rank we have two cumulative flop counts, for training and inference respectively. Also, report total number of simulation loops and any other interesting statistics that amount to utilization/useful work per process over time.

Claude Response:

I'll write the MPI/C++ simulator, then build and run it here with odd and even process counts to check the control commands (priority change, suspend/resume, both exit modes) actually behave.

While MPICH builds (the sandbox only has GitHub access, so I'm compiling MPI and OpenBLAS from source for testing), I'll write the simulator.

Both libraries are built. Compiling the simulator:

Both runs behave as specified. One label fix, then a README.

I've written the simulator as one C++/MPI file (`sim.cpp`) with a Makefile, a control helper script (`simctl.sh`) and a README. It compiles without warnings against MPICH 4.2.3 and OpenBLAS 0.3.28, which I built from source here. I tested it with 3 and 4 processes, driving the control commands live during each run, and everything behaved as you described.

Changing settings while it runs. A program's `argc`/`argv` can't change after it starts, so there is nothing new to "check" there. Instead:

- The command line sets the starting values (e.g. `p:0.5`).
- A thread on rank 0 keeps reading the same `p:`, `s:`, `r:` and `e:` commands from a control file (`./simctl.sh p:0.9`) or from the terminal you launched `mpirun` from.
- Rank 0 writes the current settings into a shared MPI-3 RMA window, and every rank reads them from there with atomic operations.

What the test runs showed:

- **Checkpoint/resume self-test (`--selftest`):** training and inference tasks that are checkpointed halfway and then resumed finish bit-for-bit identical to uninterrupted runs.
- **Priority 0.5 → 0.9:** all 3 training tasks in progress saved a checkpoint within about 4 ms and the ranks switched to inference. Each saved task was picked up again later, the next time that rank chose a training task.
- **`s:1` then `r:1`:** rank 1 saved its in-progress inference task to `checkpoints/rank_1.ckpt` and stopped taking tasks. After `r:1` it finished that saved task before taking any new ones.
- **`s:0`:** suspending rank 0 still works, even though it holds the global counter. The other ranks kept getting new tasks the whole time.
- **`e:1` then restart:** the next launch, with 4 processes, loaded the 3 saved checkpoints and finished them first.
- **`e:0`:** every rank dropped its in-progress task immediately. The report shows the dropped work as "discarded" (0.29 GFLOP in that run).

Report at exit. For each rank, it lists:

- number of loops, including idle loops while suspended;
- completed training and inference tasks;
- separate cumulative training GFLOP and inference GFLOP;
- percentage of time busy, suspended and on overhead;
- GFLOP/s while busy;
- number of preemptions, resumes and checkpoint writes.

It also gives the final task-counter value, the actual training/inference mix, and a table of busy % and GFLOP/s for each rank over 10 time windows. The full timeline is written to `sim_timeline.csv`.

Choices I made where your spec was open:

- Task data is regenerated from the task ID, so a checkpoint only stores the epoch number and weights (training) or the sample position and running checksum (inference).
- To keep the mix fair, each rank's coin flip is nudged toward `p` using both its own counts and the global counts, which are re-synced every 8 tasks.
- Random dimensions are 8–128 (`--min-dim` changes the lower bound) and inference uses 2–8 layers.
- Flop counts cover the BLAS calls only.

Build with `make OPENBLAS_DIR=<prefix>` if OpenBLAS isn't on the default path.